

DartMoQP: A Unified Framework for Mixed-Precision Quantization and Pruning of Mixture-of-Experts Models

Anonymous submission

Abstract

Mixture-of-Experts (MoE) models have become a dominant architecture for scaling large language models, but their massive parameter count makes compression essential. Existing MoE compression methods treat quantization and pruning as disjoint operations, causing catastrophic accuracy degradation at ultra-low bit widths. We uncover a key enabling property: across representative per-row and vector quantizers, quantization loss is well fit by a quadratic function in the logarithmic domain ($R^2 > 0.99$), which allows extrapolation of 0-bit loss and unifies pruning and quantization in a single loss space. Built on this insight, we propose DartMoQP, a unified mixed-precision quantization and structured pruning framework for MoE architectures operating at micro-expert granularity. Its core designs include quantizer-agnostic sensitivity metrics matched to each algorithm’s error characteristics, and globally optimized cross-expert bit allocation via dynamic programming. As a side benefit, mixed-precision allocation reduces seed-induced PPL variation on models where TurboQuant’s random rotation causes significant fluctuations. Evaluated on five MoE models across the 0.5–3 bpw range (raw weight bits), DartMoQP achieves state-of-the-art performance with particularly large gains at ultra-low bitrates: at 1 bpw, average downstream task scores rise from 0.30+ to 0.50+ on multiple models.

1 Introduction

Mixture-of-Experts (MoE) models have grown so large that even 8-bit quantization requires TBs of VRAM, putting them out of reach for consumer-grade GPUs.

Mixed-precision quantization allocates differentiated bit scheme to units of varying importance, achieving higher accuracy under the same memory budget. This is especially effective for MoE architectures, which exhibit multi-level heterogeneity—expert activation frequencies differ by orders of magnitude, and neuron importance varies sharply within experts. Existing works allocate more bits to critical experts and neurons while pruning trivial units to minimize accuracy loss under fixed bpw constraints (Xu et al. 2026; Guan et al. 2026; Hu et al. 2025; Zhang and Li 2026).

Nevertheless, existing mixed-precision methods face two fundamental bottlenecks.

The first is the long-standing separation between quantization and pruning. Prior work has pursued unification along

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

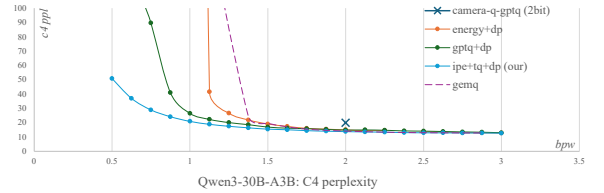


Figure 1: Experimental results (c4 perplexity) of Qwen3-30B-A3B at 0.5-3.0 bpw

multiple paths—treating 0-bit as a candidate bitwidth in AutoML search (van Baalen et al. 2020; Tang et al. 2020; Lee et al. 2025), combining static quantization with dynamic expert pruning (Huang et al. 2025, 2026), or aligning both via Hessian-aware metrics (Liu et al. 2025a; Guo, Li, and Benini 2025). Yet a fundamental obstacle remains: 0-bit loss cannot be reliably measured in the same loss space as quantization. Pruning induces a stepwise error whose magnitude depends on functional importance under the calibration distribution, not just weight magnitude, while quantization produces a gradual distortion defined on the weight manifold. These two errors live in fundamentally different spaces, making it mathematically misaligned to treat zeroing error as a 0-bit baseline. The practical consequence is that at ultra-low bit widths (<2 bpw), methods stubbornly assign 1-bit to trivial neurons that would be better off pruned entirely, leading to catastrophic accuracy degradation. A second bottleneck is that single-loss search strategies fail to adapt across quantization paradigms—in particular, weight-space metrics lose all discriminative power under vector quantizers like TurboQuant (Zandieh et al. 2025).

Through extensive pilot studies, we uncover two key properties of MoE quantization that together break the above impasses. First and most consequential, across representative per-row and vector quantizers, quantization loss is well fit by a quadratic function in the logarithmic domain, with coefficients of determination consistently above 0.99 across neurons, experts, and layers. This is not a mathematically exact theorem but a high-precision approximation in the practical bitwidth range; we provide a rate-distortion-theoretic motivation in Appendix A.1. The significance of this observation is not the fit itself, but what it enables: by extrapolating the

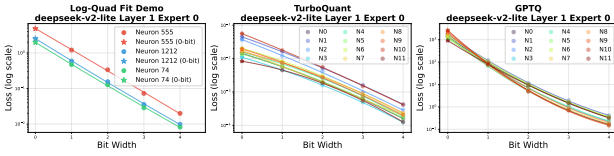


Figure 2: Log-domain quadratic fitting of quantization loss for a typical DeepSeek-v2-Lite expert.

quadratic curve to bitwidth zero, we obtain a 0-bit loss estimate residing in the same loss space as 1–4 bit measurements, resolving the misalignment between quantization error and pruning error and allowing joint optimization. Second, different quantizers require different sensitivity metrics: for vector quantizers like TurboQuant, weight-space MSE fails entirely, and input manifold-aware loss—measuring quantization error’s impact on forward outputs under calibration data—must be used instead.

Built on these insights, we propose DartMoQP, a unified mixed-precision quantization and structured pruning framework for MoE architectures operating at micro-expert granularity. Its core innovations include unified quantization-pruning modeling via 0-bit loss extrapolation, a quantizer-agnostic search framework with algorithm-matched sensitivity metrics, globally optimized cross-expert bit allocation, and reduced seed sensitivity for vector quantization via mixed-precision strategies.

Core Contributions:

1. We empirically observe that quantization loss is well fit by a quadratic function in the logarithmic domain ($R^2 > 0.99$) across representative per-row and vector quantizers, and provide a rate-distortion-theoretic motivation for this phenomenon. Combined with a simple conservative safeguard for edge-case units, this enables 0-bit loss extrapolation and a unified loss space for quantization and pruning grounded in empirical log-quadratic fitting.
2. We propose the DartMoQP framework at micro-expert granularity, supporting quantizer-agnostic mixed-precision search, fine-grained optimization, and native adaptation to vector quantizers like TurboQuant.
3. Evaluated on five MoE models across 0.5–3 bpw, DartMoQP achieves state-of-the-art performance with particularly large gains at ultra-low bitrates—at 1 bpw, average downstream task scores rise from 0.30+ to 0.50+ on multiple models. On models where TurboQuant exhibits significant seed-induced fluctuation, mixed-precision allocation also reduces PPL variation as a side benefit.

2 Related Work

2.1 Compression: Quantization or Pruning

Quantization is the primary compression technique for LLMs(Zhou et al. 2024). Per-row methods such as GPTQ(Frantar et al. 2023), AWQ(Lin et al. 2023), and SmoothQuant(Xiao et al. 2023) deliver 3-bit precision; rotation-based approaches like TurboQuant(Zandieh et al.

2025) and QuaRot(Ashkboos et al. 2024) handle outliers through orthogonal transformation.

For MoE models specifically, pruning and quantization have been studied separately. MoE pruning focuses on expert-level compression: direct and partial pruning (NAEE(Lu et al. 2024), MoEPruner(Xie et al. 2024), MoEI²(Yang et al. 2024)), expert merging or decomposition (MC-SMoE(Li et al. 2024), D2-MoE(Gu et al. 2025)), dynamic activation scheduling (Alloc-MoE(Liu et al. 2026)), with a CAMERA variant(Xu et al. 2026) pushing to fine-grained micro-expert granularity and tensor/neuron-level sparsity methods further exploiting sub-expert redundancy(Cai et al. 2026). MoE quantization targets inter-expert mixed-precision allocation via energy-based ranking (CAMERA(Xu et al. 2026)), algorithm-hardware co-design (Mx-MoE(Duanmu et al. 2025)), activation-aware and frequency-weighted methods (AFGQ(Xie et al. 2025), EAQuant(Fu et al. 2025)), global expert-level search (ExQuant(Guan et al. 2026), GEMQ(Deng et al. 2026)), routing-aware calibration (MoEQuant(Hu et al. 2025)), extreme compression (QMoE(Frantar and Alistarh 2023a)), and dynamic precision switching (FlexQuant(Liu et al. 2025b)). Vector quantizers like TurboQuant(Zandieh et al. 2025) offer new compression potential but remain under-explored for MoEs.

2.2 Unified Quantization and Pruning

Combining pruning and quantization into a single framework is a long-standing pursuit. Pruning methods such as SparseGPT(Frantar and Alistarh 2023b) establish the importance metrics that unified approaches build upon. At the system level, MC(Huang et al. 2025) and DyMoE(Huang et al. 2026) couple static mixed-precision with dynamic expert pruning; at the algorithm level, MixPQ(Liu et al. 2025a) and OBR(Guo, Li, and Benini 2025) align the two through Hessian-aware metrics; earlier works like Bayesian Bits(van Baalen et al. 2020), AJPQ(Tang et al. 2020), and AMQ(Lee et al. 2025) simply treat 0-bit as a candidate bitwidth.

All these frameworks, however, use disjoint objectives or assume a static weight space where per-element sensitivity is well-defined. None model 0-bit loss continuously within the quantizer’s own distortion-rate function—a gap that widens under vector quantizers like TurboQuant(Zandieh et al. 2025). This work bridges the gap by treating 0-bit as the natural $b = 0$ boundary of a unified log-quadratic distortion law.

3 Key Observations

Three key observations underpin DartMoQP. Additional analyses on micro-expert granularity, sensitivity metric selection, and seed sensitivity are in Appendix A.2, A.3, and A.5.

3.1 Log-Domain Quadratic Loss Law and 0-Bit Extrapolation

The central empirical finding of this work is that across representative per-row and vector quantizers, quantization loss is well fit by a quadratic function in the logarithmic domain

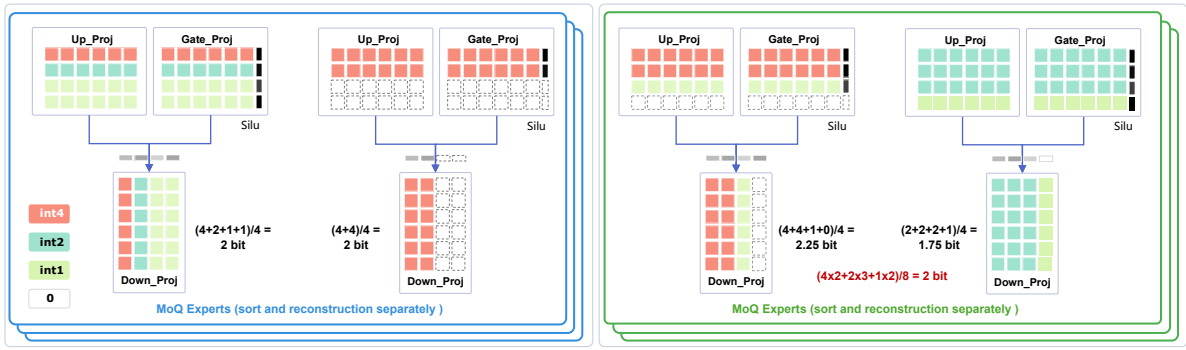


Figure 3: DartMoQP framework quantization process diagram

(Figure 2), consistently across neurons, experts, and layers:

$$\log L_i(b) = p_i b^2 + q_i b + r_i, \quad (1)$$

where b is bitwidth, $L_i(b)$ is the proxy loss of the i -th neuron or micro-expert, and p_i, q_i, r_i are fitting parameters. Coefficients of determination R^2 exceed 0.99 across all evaluated models and layers, as shown in Appendix Figure 1. This should be understood as a high-precision empirical approximation valid in the practical bitwidth range, not an exact mathematical identity; a rate-distortion-theoretic motivation is provided in Appendix A.1.

The practical significance of this law is that it enables direct extrapolation of 0-bit pruning loss as $\hat{L}_i(0) = \exp(r_i)$, placing quantization and pruning in the same continuous loss space. A simple conservative clipping safeguard is applied to a small fraction of low-sensitivity units whose extrapolated value falls below the measured 1-bit loss, ensuring the unified space is robust even for edge cases; see Appendix A.1 for detailed analysis.

3.2 Input Manifold-Aware Sensitivity for Vector Quantization

For per-row quantizers like GPTQ, weight-space metrics such as MSE or Hessian-weighted error correlate well with actual output degradation, because quantization error acts directly on the weight basis where neuron importance is naturally defined. For vector quantizers like TurboQuant, however, this assumption breaks down entirely. The core issue is TurboQuant’s random orthogonal rotation step: after rotation, weight energy is spread nearly uniformly across dimensions (geometric isotropization), so per-element MSE becomes nearly identical across neurons and loses all discriminative power for sensitivity ranking.

Crucially, this ambiguity can be resolved by anchoring the loss metric to the input distribution rather than the weight basis. Even after rotation, neurons that are more frequently activated by calibration inputs or contribute more to forward outputs remain more important. We observe that input manifold-aware metrics—which measure the projection of quantization error onto the input distribution direction—restore clear sensitivity heterogeneity across neurons, forming a reliable basis for mixed-precision allocation. A detailed comparison of sensitivity metrics is provided in Appendix A.2.

3.3 Fine-Grained Slicing Enables Cross-Neuron Budget Transfer

Mixed-precision allocation improves monotonically with slicing granularity: finer micro-experts let the search algorithm reallocate bit budget from low-sensitivity units to high-sensitivity ones, where marginal gains from larger bitwidth are substantially larger. This motivates DartMoQP’s neuron-level cross-expert search; we quantify its impact in Section 4.3.

4 DartMoQP Unified Framework

We propose DartMoQP, a unified mixed-precision quantization and structured pruning framework for MoE architectures operating at micro-expert granularity. Its core innovations include unified quantization-pruning modeling via 0-bit loss extrapolation, a quantizer-agnostic search framework with algorithm-matched sensitivity metrics, and globally optimized cross-expert bit allocation via dynamic programming. Reduced seed sensitivity on vector-quantized models emerges as a side benefit when sensitivity distributions are highly skewed.

4.1 Overall Framework Architecture

DartMoQP is a mixed-precision quantization framework for MoE architectures with global micro-expert-level bit budget reallocation, enabling cross-expert bit allocation optimization. The basic allocation unit is the micro-expert: within each expert, neurons are sorted by sensitivity and reconstructed into fixed-size micro-experts (typically 2, 4, or 8), so bitwidth can be adjusted at sub-expert granularity to exploit intra-expert heterogeneity.

DartMoQP adopts a modular pipeline: neuron-level multi-bit quantization loss via input manifold-aware evaluation; 0-bit (pruning) loss extrapolation via log-domain quadratic fitting; optimal mixed-precision search via quantizer-agnostic global dynamic programming; and deployment optimization for industrial feasibility.

Pruned units require no quantization metadata—no scales, zero points, norms, or codebook entries—unlike conventional quantization which always carries per-group overhead. This frees additional bit budget for high-sensitivity neurons, compounding the accuracy gains from unified search.

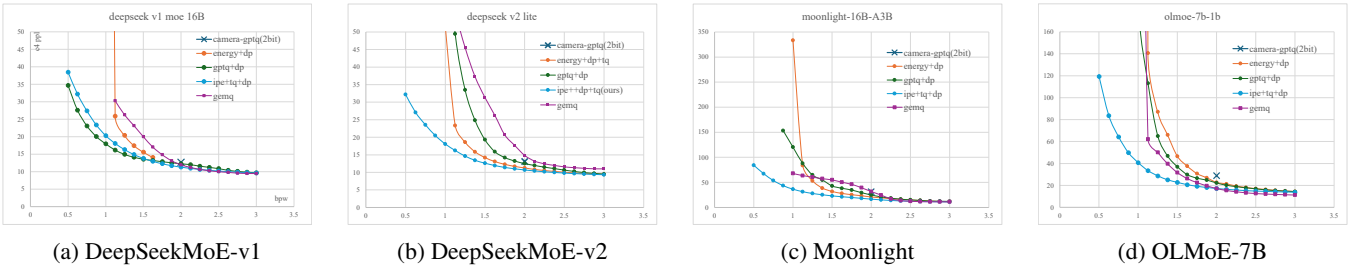


Figure 4: Experimental results of different MoE models in the range of 0.5-3.0 bpw (0.25 bpw interval)

4.2 Input Manifold-Aware Sensitivity Evaluation

DartMoQP uses input manifold-aware evaluation, which measures quantization error’s impact on forward outputs over calibration data rather than just weight-space changes. For GPTQ, we use its Hessian-aware loss; for TurboQuant, we use input manifold-aware inner product loss to handle rotation-induced sensitivity ambiguity.

Quantization Loss and Local Output Error For FFN linear projection, local output error is:

$$\begin{aligned}\mathcal{L}_{\text{out}} &= \frac{1}{n} \|XW - X\hat{W}\|_F^2 \\ &= \text{tr} \left((W - \hat{W})^\top C_X (W - \hat{W}) \right),\end{aligned}\quad (2)$$

where C_X is input covariance matrix, showing output error is weighted by input distribution.

Sensitivity Evaluation for GPTQ and TurboQuant GPTQ’s per-row quantization uses calibration input for error compensation, so we use its Hessian-aware loss. For TurboQuant, random rotation makes weight MSE ineffective, so we use input manifold-aware inner product loss:

$$\mathcal{L}_{\text{TQ}}(w_i, \hat{w}_i^b) = \mathbb{E}_{x \sim \mathcal{D}_{\text{calib}}} \left[|x^\top e_i^b| \right], \quad (3)$$

where $e_i^b = w_i - \hat{w}_i^b$ is the quantization error vector of the i -th neuron at b bits. This metric measures the projection of quantization error onto the input distribution direction, which restores sensitivity discriminability under rotation.

4.3 Quantizer-Agnostic Global Dynamic Programming Search

Direct enumeration of all bit allocation combinations is infeasible, so we design a quantizer-agnostic dynamic programming (DP) algorithm that operates at the micro-expert level.

Two design choices jointly structure the search space. First, *neuron-level sorting and micro-expert reconstruction*: within each expert, neurons are sorted by sensitivity and grouped into fixed-size micro-experts. This serves two purposes: (a) it reduces the DP state count from per-neuron to per-micro-expert, cutting search and memory costs by roughly the slice factor; (b) because neurons within a micro-expert are adjacent in the sorted sensitivity order, forcing them to share the same bitwidth introduces only negligible accuracy loss—an

ablation on slice count (Figure 7) shows diminishing returns beyond 4–8 micro-experts per expert. Second, a *non-increasing allocation constraint* along the globally sorted micro-expert sequence further reduces the search from exponential to polynomial complexity. This constraint is well-justified empirically: sensitivity rankings are largely consistent across bitwidths, so optimal allocations rarely violate monotonicity. The algorithm proceeds in four stages: within-expert neuron sorting and micro-expert reconstruction, global micro-expert ranking by sensitivity weighted with expert activation rate, DP search with non-increasing allocation constraints, and optimal scheme backtracking.

We define $dp[i][w][k]$ as the minimum cumulative loss after processing the first i micro-experts using w total bits, where the i -th micro-expert is assigned k bits. The state transition is:

$$dp[i+1][w+b][b] = \min_{b_{\text{prev}} \geq b} \left(dp[i][w][b_{\text{prev}}] + \mathcal{L}[i+1][b] \right), \quad (4)$$

where $\mathcal{L}[i][b]$ is the precomputed loss of assigning b bits to the i -th micro-expert, and the constraint $b \leq b_{\text{prev}}$ enforces non-increasing bit allocation along the sorted micro-expert sequence for deployment regularity. Further discussion is provided in Appendix A.4.

5 Experiments

5.1 Experimental Settings

We evaluate DartMoQP on five open-source MoE models: DeepSeekMoE-v1 (16B-A3B), DeepSeekMoE-v2-Lite (16B-A3B), Moonlight (16B-A3B), OLMoE-7B (7B-A1B), and Qwen3-30B-A3B. Due to space constraints, detailed results for DeepSeekMoE-v1 and OLMoE-7B are deferred to Appendix A.6.

Baseline methods. We compare against three categories of baselines: (1) *Uniform-precision baselines*: GPTQ-Origin (Frantar et al. 2023) and TQ-Origin (Zandieh et al. 2025). As static allocation methods without mixed-precision search, they only work at integer bitwidths (e.g., 1 or 2 bpw) and are not applicable to fractional targets like 1.5 bpw.

(2) *CAMERA static baselines*: CAMERA-GPTQ and CAMERA-TQ, using CAMERA’s (Xu et al. 2026) energy-based ranking for expert-level allocation without search. CAMERA-TQ is our extension to TurboQuant. The original CAMERA method is designed for and only supports 2

Model	Method	Wiki ↓	C4 ↓	Avg. ↑	ARC-C	ARC-E	PIQA	BoolQ	Wino	MNLI	Hella	MLLU
DSv2	GPTQ-Origin	142.75	210.27	0.37	0.24	0.27	0.51	0.59	0.52	0.33	0.27	0.25
DSv2	TQ-Origin	35.78	49.43	0.39	0.22	0.34	0.55	0.57	0.49	0.35	0.32	0.25
DSv2	GEMQ	36.00	70.72	0.40	0.22	0.38	0.56	0.60	0.52	0.35	0.30	0.25
DSv2	CAMERA-DP	37.79	51.51	0.38	0.23	0.32	0.55	0.57	0.50	0.34	0.31	0.26
DSv2	GPTQ-DP	59.08	100.63	0.36	0.24	0.27	0.50	0.54	0.51	0.32	0.27	0.24
DSv2	IPE-TQ-DP	8.83	18.26	0.52	0.37	0.67	0.67	0.49	0.58	0.39	0.50	0.36
Moon	GPTQ-Origin	354.38	569.41	0.36	0.24	0.31	0.53	0.45	0.50	0.34	0.28	0.25
Moon	TQ-Origin	222.65	260.44	0.38	0.23	0.33	0.55	0.56	0.50	0.35	0.30	0.26
Moon	GEMQ	33.46	67.94	0.44	0.26	0.46	0.59	0.64	0.52	0.38	0.36	0.28
Moon	CAMERA-DP	249.48	333.55	0.38	0.22	0.34	0.55	0.56	0.50	0.35	0.30	0.25
Moon	GPTQ-DP	57.33	132.22	0.39	0.22	0.33	0.55	0.55	0.50	0.36	0.31	0.26
Moon	IPE-TQ-DP	14.87	36.87	0.48	0.33	0.61	0.63	0.64	0.55	0.34	0.45	0.30
Qwen3	GPTQ-Origin	4221.86	4872.95	0.35	0.25	0.27	0.52	0.41	0.51	0.34	0.26	0.25
Qwen3	TQ-Origin	1514.08	1284.21	0.36	0.24	0.28	0.52	0.46	0.50	0.34	0.28	0.25
Qwen3	GEMQ	179.96	221.59	0.40	0.25	0.37	0.58	0.55	0.53	0.35	0.30	0.25
Qwen3	CAMERA-DP	1886.72	1422.79	0.36	0.23	0.30	0.53	0.42	0.51	0.34	0.27	0.24
Qwen3	GPTQ-DP	982.38	1798.25	0.36	0.24	0.28	0.53	0.44	0.54	0.34	0.28	0.24
Qwen3	IPE-TQ-DP	28.18	48.20	0.54	0.39	0.66	0.64	0.71	0.57	0.50	0.42	0.43

Table 1: Performance comparison of different methods on various MoE models at 1.0 bpw

bpw operation (it lacks 0-bit support), so we report these two baselines only at 2 bpw as industrial reference points.

(3) *Mixed-precision search baselines*: GEMQ(Deng et al. 2026), CAMERA-DP. GEMQ is an expert-level mixed-precision method with its own metric and GPTQ quantization. CAMERA-DP replaces our sensitivity metric with CAMERA’s energy-based ranking while retaining the same DP search framework, providing a fair apples-to-apples comparison that isolates the effect of the sensitivity metric.

Our two methods: **GPTQ-DP** (GPTQ backend with Hessian-aware loss) and **IPE-TQ-DP** (TurboQuant backend with inner-product error), both using the same global DP search with unified 0-bit modeling.

All methods use the same Wikitext-2 calibration dataset consistent with prior work, more detailed in Appendix A.7. We divide each expert into 8 micro-experts, allocate bitwidths from $\{0, 1, 2, 3, 4\}$ bits, and report perplexity (PPL) on WikiText-2 and C4 (lower is better), plus accuracy on 8 downstream tasks and their average (Avg., higher is better).

Quantization details. Both GPTQ and TurboQuant use a group size of 128. For GPTQ, each group has two FP16 quantization parameters (scale and zero point), adding 0.25 bpw of metadata. For TurboQuant, each group has one FP32 norm parameter plus shared codebooks, resulting in a metadata overhead of 0.252 bpw.

A practical advantage of unified modeling is that pruned units incur no quantization parameter overhead (no scales, zero points, or codebook entries), freeing metadata budget for higher-precision units under the same raw-weight-bit constraint. All methods are evaluated under the same raw weight bpw target, making this a genuine advantage of the unified approach rather than an unequal comparison.

All experiments run on a single NVIDIA RTX 5090 GPU

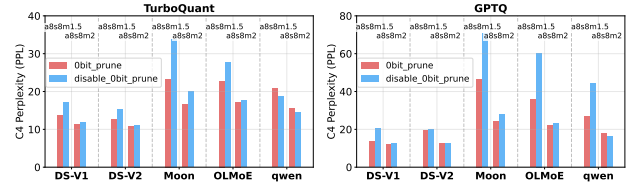


Figure 5: Impact of unified 0-bit pruning modeling on quantization PPL of different MoE models.

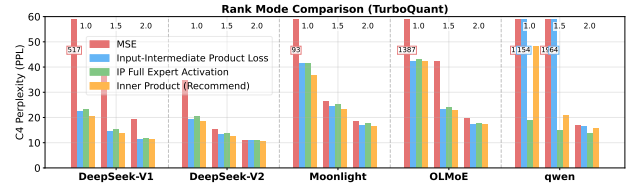


Figure 6: PPL comparison of different sensitivity metrics on C4 dataset.

with 48GB VRAM. Our layer-wise quantization process quantizes each layer independently and releases intermediate memory immediately, allowing us to quantize a 30B MoE model like Qwen3-30B-A3B in approximately 2.5 hours. Additional details on inference kernel optimization can be found in Appendix A.8.

5.2 Main Experimental Results

Figure 4 shows C4 perplexity across the 0.5–3.0 bpw range. IPE-TQ-DP consistently outperforms all baselines, with the gap widening as bitwidth decreases—confirming that unified quantization-pruning modeling delivers the largest gains precisely where existing methods struggle most. Detailed

Model	Method	Wiki ↓	C4 ↓	Avg. ↑	ARC-C	ARC-E	PIQA	BoolQ	Wino	MNLI	Hella	MMLU
DSv2	GEMQ	13.35	31.33	0.46	0.28	0.56	0.64	0.62	0.53	0.35	0.41	0.26
DSv2	CAMERA-DP	8.98	14.22	0.60	0.44	0.72	0.72	0.77	0.64	0.44	0.64	0.43
DSv2	GPTQ-DP	11.12	19.65	0.49	0.34	0.61	0.64	0.59	0.56	0.38	0.50	0.28
DSv2	IPE-TQ-DP	7.32	12.61	0.61	0.48	0.77	0.73	0.72	0.66	0.46	0.66	0.44
Moon	GEMQ	21.40	55.08	0.46	0.29	0.52	0.60	0.65	0.55	0.41	0.41	0.28
Moon	CAMERA-DP	17.36	31.79	0.52	0.36	0.64	0.68	0.67	0.57	0.38	0.51	0.34
Moon	GPTQ-DP	19.32	46.68	0.45	0.28	0.49	0.61	0.64	0.55	0.32	0.43	0.29
Moon	IPE-TQ-DP	9.99	23.27	0.55	0.43	0.71	0.71	0.65	0.57	0.35	0.58	0.43
Qwen3	GEMQ	10.91	19.50	0.62	0.52	0.81	0.71	0.82	0.63	0.44	0.54	0.50
Qwen3	CAMERA-DP	12.14	19.13	0.68	0.56	0.81	0.73	0.85	0.68	0.66	0.48	0.63
Qwen3	GPTQ-DP	16.93	26.86	0.49	0.28	0.44	0.66	0.70	0.63	0.40	0.53	0.29
Qwen3	IPE-TQ-DP	13.79	20.88	0.71	0.58	0.82	0.75	0.85	0.70	0.70	0.60	0.65

Table 2: Performance comparison of different methods on various MoE models at 1.5 bpw.

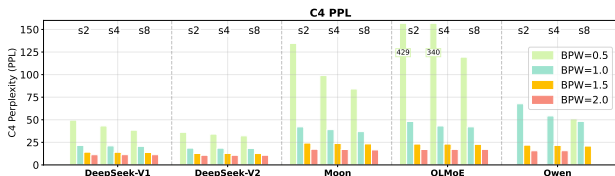


Figure 7: PPL comparison of global DP schemes on C4 with different numbers of slices. Lower values are better.

downstream task results at three representative bitwidths are reported below.

At 1.0 bpw (Table 1), IPE-TQ-DP achieves the highest average score across all models shown, with most improving from the 0.30–0.39 range to 0.47–0.54. On Qwen3-30B-A3B, the average rises from 0.355 (best baseline) to 0.539, with MMLU jumping from 0.243 to 0.432. GEMQ, lacking intra-expert pruning and 0-bit support, collapses on several models at this bitwidth. At 1.5 bpw (Table 2), IPE-TQ-DP remains dominant: Qwen3-30B-A3B reaches 0.705 average (+0.03 over CAMERA-DP), and DeepSeekMoE-v2-Lite also shows clear gains. At 2.0 bpw (Table 3), the industrial mainstream setting, IPE-TQ-DP still leads. Qwen3-30B-A3B reaches 0.757 average (MMLU 0.763), while DeepSeekMoE-v2-Lite achieves 0.667. Results for DeepSeekMoE-v1 and OLMoE-7B at all three bitwidths are provided in Appendix A.6.

These results confirm that DartMoQP’s unified quantization-pruning modeling and input manifold-aware sensitivity evaluation are effective across the full bitwidth range, with the most pronounced advantages at ultra-low bitrates.

5.3 Ablation Experiments

Unified 0-bit pruning. By extrapolating 0-bit pruning loss from the log-domain quadratic fit, DartMoQP avoids manual penalty coefficients and unifies pruning with quantization in a single loss space.

Figure 5 shows that disabling 0-bit pruning increases C4

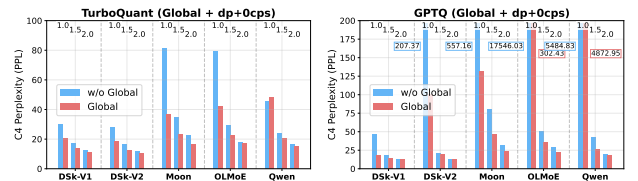


Figure 8: PPL comparison between global DP search and layer-wise independent search on C4. Lower values are better.

PPL by 15–30% at 0.5–1.5 bpw. The benefit is bitrate-dependent: at ultra-low bitrates (<1.5 bpw), pruning is essential—it frees weight bits for critical neurons that would otherwise be stuck at 1 bit, and simultaneously eliminates quantization metadata for pruned units, compounding the effective budget gain; at medium bitrates (1.5–2.5 bpw), it provides moderate gains; above 2.5 bpw, the effect diminishes as most neurons already have sufficient precision.

Input manifold-aware loss. Figure 6 compares several slice sensitivity metrics under TurboQuant, including weight MSE, input-intermediate product loss, inner product loss with activation weighting, and the inner product loss used in this paper. Replacing weight MSE with the inner product metric improves sensitivity distinguishability dramatically, raising average downstream scores by 0.15–0.25 at 1 bpw by enabling the search to reliably identify high-sensitivity neurons.

Cross-expert global DP vs. same-expert allocation. The default approach allocates uniform bitwidth within each expert based on its rank. DartMoQP instead sorts micro-experts globally across all experts (weighted by activation rate) and jointly optimizes via DP. Figure 8 shows that cross-expert global DP reduces PPL by 0.05–0.10 and raises average downstream scores by 0.03–0.08 across all bitrates, with larger gains at <2 bpw.

Micro-expert granularity. Figure 7 reports C4 PPL with 2, 4, and 8 micro-experts per expert. Finer granularity consistently yields lower PPL, confirming that neuron sensitivity

Model	Method	Wiki ↓	C4 ↓	Avg. ↑	ARC-C	ARC-E	PIQA	BoolQ	Wino	MNLI	Hella	MMLU
DSv2	GPTQ-Origin	8.88	13.53	0.58	0.41	0.71	0.74	0.67	0.61	0.41	0.69	0.40
DSv2	TQ-Origin	7.96	11.03	0.65	0.48	0.78	0.78	0.76	0.70	0.46	0.73	0.52
DSv2	CAMERA-GPTQ	8.42	13.08	0.60	0.46	0.74	0.74	0.66	0.68	0.41	0.69	0.43
DSv2	CAMERA-TQ	7.40	10.95	0.66	0.49	0.78	0.77	0.79	0.73	0.47	0.73	0.50
DSv2	GEMQ	8.47	14.82	0.57	0.42	0.74	0.76	0.66	0.63	0.40	0.61	0.37
DSv2	CAMERA-DP	7.35	11.27	0.67	0.50	0.79	0.77	0.80	0.71	0.50	0.74	0.50
DSv2	GPTQ-DP	8.11	12.58	0.61	0.45	0.76	0.75	0.69	0.66	0.40	0.70	0.46
DSv2	IPE-TQ-DP	6.78	10.69	0.67	0.52	0.81	0.78	0.78	0.69	0.51	0.73	0.52
Moon	GPTQ-Origin	14.14	31.47	0.47	0.34	0.60	0.62	0.55	0.55	0.37	0.49	0.27
Moon	TQ-Origin	15.56	23.36	0.61	0.46	0.74	0.72	0.73	0.59	0.44	0.61	0.56
Moon	CAMERA-GPTQ	11.29	26.93	0.49	0.37	0.64	0.64	0.52	0.57	0.37	0.50	0.29
Moon	CAMERA-TQ	11.61	20.79	0.62	0.48	0.76	0.74	0.74	0.59	0.43	0.63	0.57
Moon	GEMQ	10.63	31.68	0.53	0.38	0.66	0.67	0.70	0.55	0.39	0.51	0.37
Moon	CAMERA-DP	10.36	20.76	0.61	0.47	0.75	0.73	0.73	0.59	0.43	0.63	0.53
Moon	GPTQ-DP	10.17	24.14	0.52	0.38	0.65	0.66	0.64	0.54	0.39	0.53	0.39
Moon	IPE-TQ-DP	8.02	16.59	0.62	0.51	0.78	0.74	0.68	0.62	0.44	0.65	0.53
Qwen3	GPTQ-Origin	13.05	19.99	0.56	0.37	0.61	0.71	0.74	0.63	0.45	0.63	0.29
Qwen3	TQ-Origin	15.56	19.57	0.73	0.62	0.84	0.77	0.88	0.68	0.72	0.65	0.72
Qwen3	CAMERA-GPTQ	11.69	18.58	0.55	0.39	0.63	0.73	0.68	0.67	0.39	0.67	0.28
Qwen3	CAMERA-TQ	10.93	15.29	0.75	0.62	0.85	0.78	0.87	0.70	0.72	0.69	0.74
Qwen3	GEMQ	9.50	14.04	0.72	0.61	0.84	0.79	0.85	0.70	0.56	0.71	0.66
Qwen3	CAMERA-DP	10.33	14.97	0.75	0.66	0.86	0.80	0.88	0.70	0.79	0.54	0.74
Qwen3	GPTQ-DP	11.64	18.15	0.64	0.47	0.73	0.74	0.82	0.68	0.58	0.68	0.39
Qwen3	IPE-TQ-DP	10.88	15.51	0.76	0.64	0.86	0.79	0.88	0.70	0.70	0.74	0.76

Table 3: Performance comparison of different methods on various MoE models at 2.0 bpw.

Seed	Fixed-bit TQ (2 slices)		Fixed-bit TQ (8 slices)		IPE-TQ-DP (ours)	
	Wiki ↓	C4 ↓	Wiki ↓	C4 ↓	Wiki ↓	C4 ↓
0	24.30	33.81	23.94	33.59	7.33	11.44
42	11.76	16.50	11.62	16.67	7.28	11.46
84	13.47	22.13	13.32	22.09	7.35	11.47
126	15.61	23.11	15.65	23.04	7.30	11.42
168	21.98	32.43	21.30	32.16	7.32	11.45
210	11.72	18.98	11.61	18.72	7.32	11.48

Table 4: Effect of TurboQuant rotation seeds on quantization stability (DeepSeek-MoE-16B, 2.0 raw bpw).

within experts is non-uniform. We use 8 micro-experts per expert as the default for the best accuracy–efficiency trade-off.

Seed sensitivity stabilization. TurboQuant’s random rotation introduces seed-dependent variation. On DeepSeek-MoE-16B, where baseline fluctuation is pronounced, Table 4 shows that IPE-TQ-DP reduces the C4 PPL range from 16.50–33.81 to 11.42–11.52 across 6 seeds by protecting the high-sensitivity units that drive seed-induced variation. More discussion is in Appendix A.5.

6 Conclusion

This paper proposes DartMoQP, a unified framework for mixed-precision quantization and structured pruning designed natively for MoE. The core insight is that quantization error follows a predictable log-quadratic distortion-rate trajectory, allowing 0-bit loss to be extrapolated as a natural boundary rather than treated as a separate problem. While not necessarily the optimal formulation, it offers an effective path toward a genuinely unified search framework where quantization and pruning compete on the same loss landscape. We hope this perspective inspires future compression methods that treat pruning as a continuous extension of quantization.

References

- Ashkboos, S.; Mohtashami, A.; Croci, M. L.; Li, B.; Jaggi, M.; Alistarh, D.; Hoefler, T.; and Hensman, J. 2024. QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs. ArXiv:2404.00456 [cs].
- Cai, W.; Qin, L.; He, S.; Cui, J.; Li, A.; and Huang, J. 2026. Mining Tensor/Neuron-Level Sparsity to Maximize Mixture-of-Experts Potential in Post-Training and Inference. In *Forty-third International Conference on Machine Learning*.
- Deng, J.; Wang, S.; Wang, D.; Liu, Z.; Chen, T.; Yang, H.; and Hu, J. 2026. GEMQ: Global Expert-Level Mixed-Precision Quantization for MoE LLMs. In *Proceedings of the 43rd International Conference on Machine Learning*. ArXiv:2605.23078.

- Duanmu, H.; Li, X.; Yuan, Z.; Zheng, S.; Duan, J.; Zhang, X.; and Lin, D. 2025. MxMoE: Mixed-precision Quantization for MoE with Accuracy and Performance Co-Design. In *International Conference on Machine Learning*. Vancouver, Canada: arXiv. ArXiv:2505.05799 [cs].
- Frantar, E.; and Alistarh, D. 2023a. Qmoe: Practical sub-1-bit compression of trillion-parameter models. *arXiv preprint arXiv:2310.16795*.
- Frantar, E.; and Alistarh, D. 2023b. SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot. ArXiv:2301.00774 [cs].
- Frantar, E.; Ashkboos, S.; Hoefler, T.; and Alistarh, D. 2023. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. ArXiv:2210.17323 [cs].
- Fu, Z.; Ding, N.; Han, K.; Yu, X.; Li, X.; Chen, X.; Tang, Y.; and Wang, Y. 2025. EAQuant: Enhancing Post-Training Quantization for MoE Models via Expert-Aware Optimization. ArXiv:2506.13329 [cs].
- Gu, H.; Li, W.; Li, L.; and others. 2025. Delta Decompression for MoE-based LLMs Compression. *International Conference on Machine Learning*, 20497–20514.
- Guan, C.; Liu, F.; Wang, J.; Yang, N.; and Guan, H. 2026. ExQuant: Globally Comparable Expert Importance for Mixed-Precision Post-Training Quantization. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*.
- Guo, H.; Li, Y.; and Benini, L. 2025. Optimal brain restoration for joint quantization and sparsification of llms. *arXiv preprint arXiv:2509.11177*.
- Hu, X.; Chen, Z.; Yang, D.; Xu, Z.; Xu, C.; Yuan, Z.; Zhou, S.; and Yu, J. 2025. MoEQuant: Enhancing Quantization for Mixture-of-Experts Large Language Models via Expert-Balanced Sampling and Affinity Guidance. ArXiv:2505.03804 [cs].
- Huang, W.; Liao, Y.; Liu, J.; He, R.; Tan, H.; Zhang, S.; Li, H.; Liu, S.; and Qi, X. 2025. Mixture compressor for mixture-of-experts llms gains more. In *International Conference on Learning Representations*, volume 2025, 68628–68648.
- Huang, Y.; Fang, Z.; Luo, W.; Wu, R.; Chen, W.; and Zheng, Z. 2026. DyMoE: Dynamic Expert Orchestration with Mixed-Precision Quantization for Efficient MoE Inference on Edge. ArXiv:2603.19172 [cs.LG].
- Lee, S.; Woo, S.-t.; Jin, J.; Lee, C.; and Park, E. 2025. AMQ: Enabling AutoML for Mixed-precision Weight-Only Quantization of Large Language Models. ArXiv:2509.12019 [cs].
- Li, P.; Zhang, Z.; Yadav, P.; Sung, Y.-L.; Cheng, Y.; Bansal, M.; and Chen, T. 2024. Merge, then compress: Demystify efficient smoe with hints from its routing policy. In *International Conference on Learning Representations*, volume 2024.
- Lin, J.; Tang, J.; Tang, H.; Yang, S.; Dang, X.; Gan, C.; and Han, S. 2023. AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. ArXiv:2306.00978 [cs].
- Liu, B.; Gu, T.; Wang, H.; and Qian, Y. 2025a. MixPQ: Joint Pruning and Quantization for Speech and Language Foundation Models Compression. *IEEE Transactions on Audio, Speech and Language Processing*, 33: 4098–4112.
- Liu, B.; Tian, K.; Wang, W.; Zhang, Z.; Qiao, L.; and Li, D. 2026. Alloc-MoE: Budget-Aware Expert Activation Allocation for Efficient Mixture-of-Experts Inference. ArXiv:2604.08133 [cs].
- Liu, F.; Wang, Z.; Xia, J.; Zhao, J.; Zhao, S.; Li, J.; Liu, J.; Jiang, L.; and Guan, H. 2025b. FlexQuant: A Flexible and Efficient Dynamic Precision Switching Framework for LLM Quantization. ArXiv:2506.12024 [cs.LG].
- Lu, X.; Liu, Q.; Xu, Y.; Zhou, A.; Huang, S.; Zhang, B.; Yan, J.; and Li, H. 2024. Not All Experts are Equal: Efficient Expert Pruning and Skipping for Mixture-of-Experts Large Language Models. In Ku, L.-W.; Martins, A.; and Srikumar, V., eds., *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 6159–6172. Bangkok, Thailand: Association for Computational Linguistics.
- Tang, W.; et al. 2020. Automated Model Compression by Jointly Applied Pruning and Quantization. *arXiv preprint arXiv:2011.06231*.
- van Baalen, M.; Louizos, C.; Nagel, M.; Amjad, R. A.; Wang, Y.; Blankevoort, T.; and Welling, M. 2020. Bayesian Bits: Unifying Quantization and Pruning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33.
- Xiao, G.; Lin, J.; Seznec, M.; Wu, H.; Demouth, J.; and Han, S. 2023. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. ArXiv:2211.10438 [cs].
- Xie, Y.; Zhang, Z.; Zhou, D.; Xie, C.; Song, Z.; Liu, X.; Wang, Y.; Lin, X.; and Xu, A. 2024. Moe-pruner: Pruning mixture-of-experts large language model using the hints from its router. *arXiv preprint arXiv:2410.12013*.
- Xie, Z.; Ma, Y.; Zheng, X.; Chao, F.; Sui, W.; Li, Y.; Li, S.; and Ji, R. 2025. Automated Fine-Grained Mixture-of-Experts Quantization. In *Findings of the Association for Computational Linguistics: ACL 2025*, 27024–27037.
- Xu, Y.; Han, X.; Zhang, Y.; Wang, Y.; Liu, Y.; Ji, S.; Zhu, Q.; and Che, W. 2026. Camera: Multi-matrix joint compression for moe models via micro-expert redundancy analysis. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, 27395–27404.
- Yang, C.; Sui, Y.; Xiao, J.; Huang, L.; Gong, Y.; Duan, Y.; Jia, W.; Yin, M.; Cheng, Y.; and Yuan, B. 2024. MoE-IS²: Compressing Mixture of Experts Models through Inter-Expert Pruning and Intra-Expert Low-Rank Decomposition. ArXiv:2411.01016 [cs].
- Zandieh, A.; Daliri, M.; Hadian, M.; and Mirrokni, V. 2025. TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate. ArXiv:2504.19874.
- Zhang, Z.; and Li, Z. 2026. DMQ: Direct Mixed-Precision Quantization for Fixed-Memory-Constrained GPU LLM Deployment. In *International Conference on Intelligent Computing*, 2026.
- Zhou, Z.; Ning, X.; Hong, K.; Fu, T.; Xu, J.; Li, S.; Lou, Y.; Wang, L.; Yuan, Z.; Li, X.; Yan, S.; Dai, G.; Zhang, X.-P.; Dong, Y.; and Wang, Y. 2024. A Survey on Efficient Inference for Large Language Models. ArXiv:2404.14294 [cs].

A Appendix

A.1 Log-Domain Quadratic Law and 0-Bit Extrapolation

We compute per-neuron proxy quantization loss at 1–4 bits using the sensitivity metric matched to each quantizer, and examine its dependence on bitwidth in the logarithmic domain. A consistent empirical pattern emerges:

$$\log L_i(b) = p_i b^2 + q_i b + r_i, \quad (1)$$

where $L_i(b)$ is the loss of the i -th neuron or micro-expert at bitwidth b , and p_i, q_i, r_i are fitting parameters.

Figure 2 shows representative curves for a DeepSeek-v2-Lite expert: both shallow and deep layers are well fit by a quadratic function, with different neurons exhibiting consistent shapes but shifted vertically according to their sensitivity. The pattern holds across GPTQ and TurboQuant backends, across neurons with widely varying sensitivity within the same expert, and across both shallow and deep network layers.

Figure 1 summarizes the fitting goodness across all evaluated models and layers, with median R^2 values near 0.99, confirming that the quadratic fit is not an isolated observation but a stable statistical regularity.

The practical value of this empirical law is that it enables direct extrapolation of 0-bit (pruning) loss as

$$\hat{L}_i(0) = \exp(r_i), \quad (2)$$

placing pruning loss in the same loss space as 1–4 bit quantization and eliminating the need for manual pruning penalty coefficients.

Rate-Distortion Theoretic Motivation The log-domain quadratic fit is an *empirical high-precision approximation* valid in the practical bitwidth range (roughly $b \in [1, 6]$), not a mathematically exact theorem. Below we provide a rate-distortion-theoretic motivation for why such a form emerges naturally.

High-bit regime. In the high-resolution limit, the Panter-Dite formula gives the distortion of a source with density $f(x)$ under b -bit scalar quantization as

$$D(b) \approx \frac{1}{12} \left(\int |f(x)|^{1/3} dx \right)^3 \cdot 4^{-b},$$

so $\log D(b) = \log C - 2b \log 2$ —a linear function of b , corresponding to a quadratic coefficient $p = 0$.

Low-bit regime. At low bitrates the high-resolution approximation breaks down, introducing correction terms:

$$D(b) = C_0 \cdot 4^{-b} \cdot (1 + C_1 \cdot 2^{-b} + C_2 \cdot 4^{-b} + o(4^{-b})).$$

Taking logarithms and expanding $\log(1 + x) \approx x - x^2/2$ introduces terms in 2^{-b} and 4^{-b} . Over the limited interval $b \in [1, 6]$, this smooth function can be approximated by a quadratic polynomial $pb^2 + qb + r$ with extremely high precision.

Robustness of 0-Bit Extrapolation The log-domain quadratic fit holds for the vast majority of neurons with $R^2 > 0.99$, forming the empirical foundation of our unified quantization-pruning framework. Nevertheless, extrapolation to $b = 0$ is not guaranteed to be accurate for every individual unit, and a subset of neurons exhibit non-smooth behavior in the 1–2 bit regime that can make $\hat{L}_i(0) = \exp(r_i)$ unreliable.

Our position on this issue is pragmatic and three-fold. First, we quantify the problem: violations are rare overall, strongly concentrated in low-sensitivity and rarely-activated units, and modest in severity where they occur. Second, we apply a simple conservative safeguard—a clipping rule with $\alpha = 2.0$ —that prevents any single unit from grossly overestimating the benefit of pruning. Third, we argue that the global DP search is inherently robust to these small perturbations, because allocation decisions depend on relative marginal gains across thousands of micro-experts rather than precise values for any single unit. The subsections below substantiate each of these claims with empirical data, and we also flag the Qwen3-30B outlier as an interesting direction for future investigation.

Prevalence of unreliable extrapolations. We flag an extrapolation as unreliable when $\hat{L}_i(0) < L_i(1)$, meaning the extrapolated 0-bit loss falls below the measured 1-bit loss, which would overestimate the benefit of pruning.

The prevalence depends strongly on the quantization backend. Under TurboQuant, the fraction of unreliable neurons ranges from 0.24% to 4.09% across the five evaluated models, with a median of approximately 0.3%. Under GPTQ, violations are negligible—less than 0.01% across all models, and in some cases fewer than ten neurons in total. This gap is consistent with the intuition that GPTQ’s Hessian-aware optimization produces smoother loss curves, while TurboQuant’s random rotation injects additional noise that can distort fits for near-flat neurons.

The vast majority of unreliable extrapolations are concentrated in low-sensitivity units. For example, in Moonlight-16B under TurboQuant, the violation rate decreases monotonically from 5.52% in the lowest sensitivity decile to 1.03% in the highest, a roughly five-fold difference. This confirms that low-sensitivity neurons—whose loss curves are nearly flat in the 1–4 bit range—are most susceptible to extrapolation error.

A second, even stronger, axis of concentration is expert activation frequency, or *expert hotness*. Intuitively, experts that are rarely activated accumulate less gradient signal during training and tend to have flatter, noisier loss curves, making them more prone to extrapolation failure. Table 1 breaks down violation rates by expert activation quintile (Q1 = coldest, Q5 = hottest) for all five models under TurboQuant. The gap is substantially larger than the sensitivity-decile gap: for Qwen3-30B-A3B, the coldest quintile has a 16.6% violation rate versus 0.5% for the hottest—a $33\times$ difference. Moonlight and OLMoE show similar patterns with $7\times$ and $12\times$ gaps, respectively. Since cold experts also tend to have lower overall sensitivity, this finding reinforces the same underlying mechanism from a different angle.

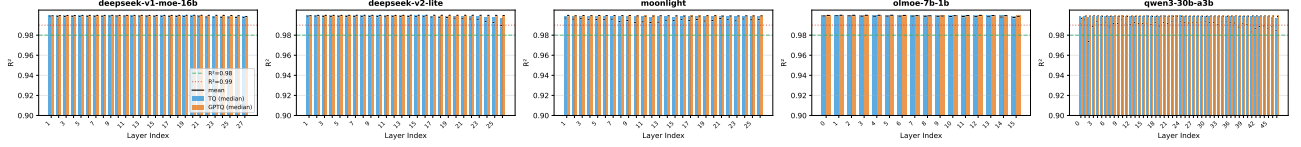


Figure 1: Comparison of log-domain quadratic fitting goodness-of-fit R^2 on different models and network layers

Model	Q1 (cold)	Q2	Q3	Q4	Q5 (hot)
		violation rate (%)			
DSMoE-V1	0.484	0.323	0.369	0.279	0.195
DSV2-Lite	1.706	1.469	1.416	1.162	0.827
Moon	7.110	2.522	2.413	1.340	0.959
OLMoE	0.746	0.274	0.093	0.076	0.064
Qwen3	16.639	4.453	1.593	0.961	0.509

Table 1: Violation rate by expert activation quintile (TurboQuant). Q1 = coldest experts (lowest activation rate), Q5 = hottest. *Note:* Experts are ranked by activation rate per layer and grouped into quintiles.

The severity of violations is also modest for most models: the median ratio $\hat{L}_i(0)/L_i(1)$ among unreliable units ranges from 0.81 to 0.87 across four of the five models, meaning the extrapolation underestimates pruning loss by 13–19% on average. Qwen3-30B-A3B is an outlier, with a median ratio of 0.63 and the highest overall violation rate (4.09%), suggesting that larger or deeper models may exhibit more complex distortion-rate behavior in the low-bit regime. We return to this point below.

Causes. Two factors contribute to unreliable extrapolations. First, low-sensitivity neurons with nearly flat loss curves in the 1–4 bit range are susceptible: small measurement noise can distort the quadratic fit near the $b = 0$ boundary. Second, neurons with bimodal or heavy-tailed activation patterns may have genuinely non-smooth distortion-rate behavior that the quadratic model cannot capture. The strong concentration of violations in low-sensitivity deciles supports the first factor as the dominant cause.

The pattern also varies substantially across layers within a single model. Table 2 reports per-stage violation rates for all five models under TurboQuant, where stages are defined by equally partitioning layers by index into early (first third), middle (middle third), and late (remaining layers). Two patterns stand out. First, the full-layer range can be very wide—from 0.04% to 10.59% in the extreme case of Qwen3-30B-A3B—indicating that depth and layer-specific activation statistics play a non-trivial role. Second, the *shape* of the depth profile differs across architectures: DeepSeek-V1/V2-Lite and OLMoE show a pronounced late-stage increase, while Moonlight is fairly flat across stages, and Qwen3-30B is high throughout with a late peak. This architectural variability suggests that layer depth alone is not the driver; rather, it interacts with each model’s specific training dynamics and activation statistics.

Model	Early	Middle	Late	Full-layer range
	violation rate (%)			
DSMoE-V1	0.15	0.11	0.73	0.04–1.54
DSV2-Lite	0.19	0.25	3.37	0.11–9.22
Moon	2.45	3.26	2.62	1.40–4.51
OLMoE	0.13	0.09	0.47	0.05–0.87
Qwen3	3.97	3.10	5.21	2.20–10.59

Table 2: Violation rate by model stage (TurboQuant). Stages are obtained by equally dividing layers by index into early (first $n/3$), middle (next $n/3$), and late (remaining).

Safeguard. As a guard against unreliable extrapolations, we apply a simple clip: if $\hat{L}_i(0) < L_i(1)$, we set $\hat{L}_i(0) = L_i(1) \cdot \alpha$, where $\alpha = 2.0$ is a conservative safety margin. This ensures pruning is never artificially cheaper than 1-bit quantization by more than a factor of two; since the affected units are predominantly low-sensitivity, the exact value of α has negligible impact on the final allocation. The safeguard affects approximately 0.2–4.1% of neurons under TurboQuant (matching the violation rates above) and effectively zero under GPTQ.

Since unreliable extrapolations are rare and predominantly affect low-sensitivity units, the global DP search naturally absorbs these small perturbations—allocation decisions depend on relative marginal gains across thousands of micro-experts, not precise values for any single unit.

Validation against direct zeroing loss. We further validate the extrapolated 0-bit loss against directly measured local zeroing loss. For each micro-expert, $L_{0,i}^{\text{direct}}$ is obtained by zeroing the unit while keeping all other weights fixed, and $\hat{L}_{0,i}^{\text{fit}}$ is the log-quadratic extrapolation from measured 1–4 bit losses.

The two scores are highly consistent in both distribution and ranking. The mean Spearman rank correlation across models and layers is 0.966:

$$\rho_S = \text{corr} \left(\text{rank} \left(\hat{L}_0^{\text{fit}} \right), \text{rank} \left(L_0^{\text{direct}} \right) \right).$$

Defining the log residual as

$$r_i = \log \hat{L}_{0,i}^{\text{fit}} - \log L_{0,i}^{\text{direct}},$$

and the robust MAD-normalized deviation

$$z_i^{\text{MAD}} = \frac{|r_i - \text{median}(r)|}{\sigma_{\text{MAD}}},$$

where $\sigma_{\text{MAD}} = 1.4826 \cdot \text{MAD}(r)$ and the factor 1.4826 calibrates the MAD to the standard deviation of a Gaussian

distribution. Only 4.06% of units exceed the MAD outlier threshold of three. Thus the extrapolated score preserves the relative unit importance reflected by direct local zeroing loss.

Despite this strong rank consistency, the extrapolated score is systematically larger than the direct score:

$$\text{median}_{i \in l} \left(\frac{\hat{L}_{0,i}^{\text{fit}} - L_{0,i}^{\text{direct}}}{L_{0,i}^{\text{direct}}} \right) \in [61.3\%, 63.3\%],$$

and more than 99.7% of units satisfy $\hat{L}_{0,i}^{\text{fit}} > L_{0,i}^{\text{direct}}$. Therefore, extrapolation should not be interpreted as an unbiased estimate of isolated zeroing loss; it provides a rank-consistent but more conservative 0-bit pruning penalty for global bit allocation.

This conservative penalty shifts the pruning–precision trade-off. We define the fraction of retained micro-experts (coverage) as

$$\text{coverage} = \frac{\#\{i : b_i > 0\}}{\#\{i\}},$$

where b_i is the bitwidth assigned to the i -th micro-expert, and the average bitwidth of retained units as

$$\text{mean surviving bits} = \frac{\sum_i b_i}{\#\{i : b_i > 0\}}.$$

Under the same total bit budget, extrapolation increases coverage while reducing the average bitwidth of retained units. On DeepSeekMoE at 1.50 bpw, for example, switching from Direct to Extrapolate increases coverage from 0.582 to 0.736, while reducing the average precision of retained units from 2.772 to 2.146 bits. Thus Direct favors aggressive pruning with concentrated precision, whereas Extrapolate retains more FFN hidden-channel directions at lower precision.

Which strategy is preferable depends on the target bitrate. At 0.75 bpw, Direct outperforms Extrapolate because aggressive pruning concentrates the scarce budget on the most sensitive units. The crossover occurs early, however: on DeepSeek-V2-Lite, Extrapolate already outperforms Direct at 1.0 bpw (C4 PPL of 18.18 vs. 18.23) and maintains a clear advantage at higher bitrates. This suggests that once the budget is sufficient to retain additional low-to-medium sensitivity units, conservative extrapolation avoids excessive removal of FFN hidden-channel directions and yields better end-to-end performance.

Further discussion on the Qwen3-30B outlier. Qwen3-30B-A3B exhibits both the highest violation rate and the most severe underestimation among the five models. While this could be due to its larger size (30B parameters vs. 7–16B for the others), it may also reflect architectural differences such as wider experts, different activation functions, or different depth-wise sensitivity distributions. A more systematic investigation—varying model size while controlling architecture, or examining per-layer patterns across model scales—would be needed to disentangle these factors. We leave this as an interesting direction for future work.

A.2 Sensitivity Metrics for Different Quantizers

Different quantizers have different error characteristics in MoE low-bit compression, and no single sensitivity metric

works equally well across all of them. DartMoQP uses a unified search framework but matches different sensitivity metrics to different quantizer backends.

Why Per-Element MSE Fails for Rotating Quantizers

TurboQuant’s random orthogonal matrix transformation introduces geometric isotropization: random orthogonal transforms approximate Haar measure properties, flattening the energy distribution of neuron weight vectors across dimensions and making them nearly isotropic in the new basis. After rotation, weight component errors become statistically identical across neurons, so traditional per-element weight MSE loses its ability to distinguish neuron importance, and the static numerical reconstruction error it measures no longer correlates with actual model output impact. As shown in Figure 2 (c), TurboQuant evaluated with per-element MSE yields highly convergent losses and nearly uniform allocation, leaving no room for mixed-precision optimization.

Input Manifold-Aware Loss To address this issue, we need a loss metric that measures the impact of quantization errors on actual model outputs rather than just static weight reconstruction. The core insight is that neurons activated more frequently by inputs or contributing more to outputs will have larger quantization losses; this principle holds even after rotation, because the input manifold preserves the relative importance of neurons.

For a linear projection in an FFN, the local output error takes the form

$$\begin{aligned} \mathcal{L}_{\text{out}} &= \frac{1}{n} \|XW - X\hat{W}\|_F^2 \\ &= \text{tr} \left((W - \hat{W})^\top C_X (W - \hat{W}) \right), \end{aligned} \quad (3)$$

where $C_X = \frac{1}{n} X^\top X$ is the empirical covariance matrix of the calibration set input. This formula shows that the impact of weight quantization error on the output is weighted by the input distribution.

The two quantizers we support both fall under this input-manifold-aware framework, differing only in the specific loss functional form:

TurboQuant (L1 inner-product loss). For the rotating quantizer, we define the sensitivity metric as the expected absolute inner product error:

$$\mathcal{L}_{\text{TQ}}(w_i, \hat{w}_i^b) = \mathbb{E}_{x \sim \mathcal{D}_{\text{calib}}} [|x^\top e_i^b|], \quad (4)$$

where $e_i^b = w_i - \hat{w}_i^b$ is the quantization error of the i -th neuron at bitwidth b . This loss directly measures the projection magnitude of the quantization error onto the calibration input directions.

GPTQ (L2 Hessian-aware loss). For non-rotating per-row quantizers like GPTQ, the quantization process already uses calibration input to estimate Hessian information. We therefore directly use the Hessian-aware loss as the sensitivity source:

$$\mathcal{L}_{\text{GPTQ}}(w_i, \hat{w}_i^b) = \mathbb{E}_{x \sim \mathcal{D}_{\text{calib}}} \left[(x^\top e_i^b)^2 \right] = (e_i^b)^\top C_X e_i^b, \quad (5)$$

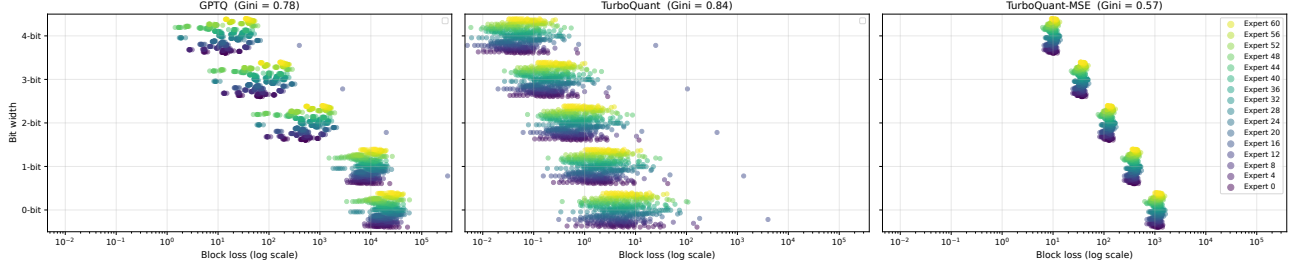


Figure 2: Comparison of micro-expert-level quantization loss for layer 1 of DeepSeek-v1-MoE-16B under different quantizers and loss metrics

which is consistent with the local output error in Equation 3.

Both metrics express the same underlying idea—weighting quantization error by its impact on forward outputs through the input distribution—and both feed into the same global DP search. Figures 2 (a) and (b) verify the effectiveness of this approach for TurboQuant and GPTQ respectively, showing significant sensitivity heterogeneity that provides clear optimization space for mixed-precision allocation. In this way, DartMoQP does not rely on a specific quantizer but provides a unified mixed-precision search interface for different backends.

A.3 Micro-Expert Granularity and Budget Transfer

For mainstream MoE models and quantization algorithms, increasing slicing granularity reduces layer-wise DP search loss under continuous micro-expert constraints, with gains saturating at fine granularities (Figure 3). The underlying mechanism is bitwidth budget transfer: because neurons differ widely in quantization sensitivity, reducing bitwidth on low-sensitivity units costs little, while increasing bitwidth on high-sensitivity units gains much; finer granularity allows the DP search to match bitwidth to marginal gain more precisely, lowering total loss at the same average bitwidth until the gains converge.

From a complexity standpoint, the globally optimal unconstrained allocation over N neurons is infeasible to enumerate directly. The continuous micro-expert constraint—sorting neurons within each expert by sensitivity and then allocating bitwidths in non-increasing order across consecutive blocks—strikes a pragmatic balance. Strict optimality of sorted allocations requires that sensitivity rankings be consistent across all bitwidths, which rarely holds exactly; but sorting by 2-bit loss yields approximate consistency with only minor local crossovers, so the empirical cost of the constraint is small while reducing the search from exponential to polynomial.

Dynamic programming is necessary within this structured space because marginal gains vary non-uniformly across bitwidth levels. A single sensitivity ranking cannot capture which units benefit most from low-bit versus high-bit increases, whereas DP compares all combinations to find the optimal transfer of budget from low- to high-marginal-gain

units.

Budget Transfer Mechanism: An Intuitive Verification

Figure 4 provides a direct visualization of the budget transfer mechanism on four experts from the first layer of Qwen3-30B-A3B. Neurons are sorted along the horizontal axis in descending order of sensitivity, with the cyan curve marking the uniform 2-bit baseline at an average of 2.0 bpw. Starting from this baseline, we transfer bitwidth budget while keeping the global average fixed: the top 12.5% most sensitive neurons are promoted to 3 bits (orange scatter points), the bottom 12.5% least sensitive neurons are demoted to 1 bit (light green scatter points), and the middle 75% remain at 2 bits.

The result confirms the asymmetric gain pattern predicted above: high-sensitivity neurons see large loss reductions when upgraded, while low-sensitivity neurons incur only modest loss increases when downgraded, so the net effect across all four experts is a total loss reduction ranging from 18.5% to 66.9%, despite no change in average bitwidth. The following section presents the dynamic programming algorithm that implements this budget transfer principle at scale under structured micro-expert constraints.

A.4 Quantizer-Agnostic Global Dynamic Programming Search

Building on the continuous micro-expert framework established in the previous section, we now describe the dynamic programming search algorithm that finds the optimal mixed-precision allocation within this structured space. Direct enumeration of the full allocation space grows as K^T where T is the total number of micro-experts and K the number of candidate bitwidths—exponential and completely infeasible at scale. The sorted, non-increasing constraint derived above prunes the search to polynomial complexity while incurring only negligible empirical cost.

Dynamic Programming Search Algorithm The algorithm proceeds in two phases: preprocessing and DP search.

In preprocessing, neurons within each expert are sorted in descending order of sensitivity and divided into fixed-size micro-experts. All micro-experts are then ranked globally by a combined importance score—sensitivity multiplied by expert activation rate—so that units with the greatest impact

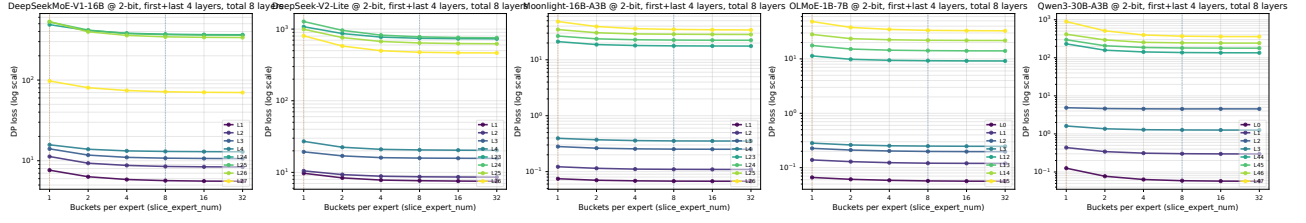


Figure 3: Layer-wise DP search loss as a function of the number of micro-experts per expert (S), shown for the first and last layers of DeepSeek-V2-Lite under TurboQuant (L1 inner-product loss). Gains saturate as S increases, confirming that moderate slicing already captures most of the available headroom.

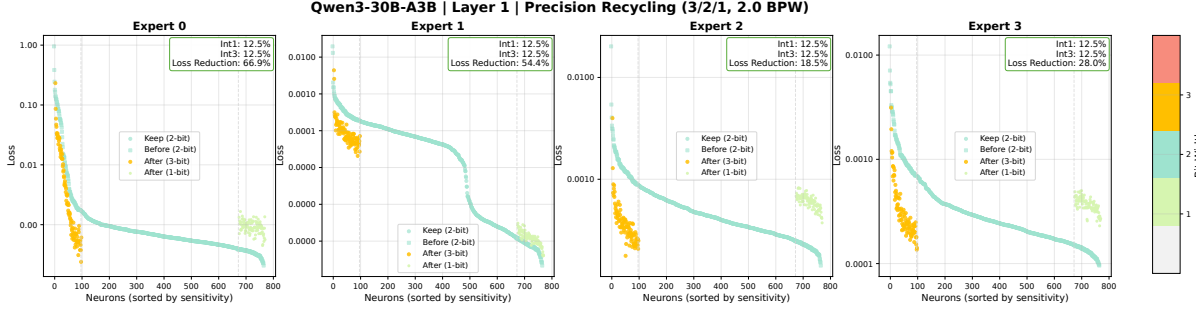


Figure 4: Budget transfer illustration on four experts from the first layer of Qwen3-30B-A3B (TurboQuant, L1 inner-product loss). Neurons are sorted by descending sensitivity. Starting from a uniform 2-bit baseline (cyan curve), promoting the top 12.5% most sensitive neurons to 3 bits (orange) and demoting the bottom 12.5% to 1 bit (green) reduces total loss while keeping the average bitwidth unchanged.

on model output are prioritized for bit allocation. This global ordering enables a deployment-friendly structured search: bitwidths are constrained to be non-increasing along the sorted order, which means within each expert the highest-sensitivity neurons receive the most bits and the lowest-sensitivity neurons receive the fewest (or zero).

The DP state is defined as

$$dp[i][w][b_{\text{prev}}] = \min_{\text{scheme}} \sum_{k=1}^i \text{block_loss}[k][b_k], \quad (6)$$

where i indexes the micro-experts processed so far, w is the consumed *raw-weight* bit budget, b_{prev} is the bitwidth assigned to the i -th micro-expert, and $\text{block_loss}[k][b_k]$ is the manifold-aware loss when the k -th micro-expert uses b_k bits. All micro-experts within a layer have the same number of hidden channels; therefore, after normalizing the raw-weight budget by the number of weights per micro-expert, assigning b bits to one micro-expert incurs a normalized cost of b .

The state transition is

$$dp[i+1][w+b][b] = \min(dp[i+1][w+b][b], dp[i][w][b_{\text{prev}}] + \text{block_loss}[i+1][b]), \quad (7)$$

subject to $b \leq b_{\text{prev}}$, which enforces the non-increasing allocation constraint. The optimal scheme is recovered by backtracking from the state with minimum loss at the target raw-weight budget.

The DP optimizes the raw-weight budget used to define the reported target bpw. Quantization metadata, including scales, zero points, and norms, is accounted for separately when reporting effective storage cost. Because a pruned micro-expert has neither quantized weights nor associated metadata, pruning reduces both components of storage.

The overall time complexity is $O(TWK^2)$, where T is the total number of micro-experts, W is the normalized target raw-weight budget, and K is the number of candidate bitwidths.

Design rationale. The non-increasing allocation constraint serves two purposes. First, it produces structurally regular allocations that simplify deployment: within each expert, bitwidths decrease monotonically along the sorted neuron order, matching the micro-expert tiling pattern expected by efficient inference kernels. Second, it prunes the search space from an unconstrained knapsack to a structured problem, making DP tractable at scale. The empirical cost of this constraint is small, as discussed in the previous section: sorting by sensitivity yields approximately optimal allocations with only minor local crossovers.

Micro-expert vs. per-neuron allocation. A natural question is why the DP operates on micro-experts rather than individual neurons. In principle, per-neuron DP is also polynomial ($O(NWK^2)$, where N is the total number of neurons) and permits finer-grained allocation. In practice, micro-expert aggregation provides an effective balance between search cost and allocation accuracy.

First, *computational and memory cost*: a typical MoE layer contains thousands of hidden channels distributed across dozens of experts. The state count of per-neuron DP grows linearly with the number of hidden channels, making layer-wise search slow and memory-intensive. After sorting neurons by sensitivity within each expert, we partition each expert into S contiguous micro-experts, where $S \in \{2, 4, 8\}$ denotes the number of micro-experts per-expert and $S = 8$ is the default. For an expert with hidden dimension d_{ff} , each micro-expert contains d_{ff}/S hidden channels. Relative to per-neuron allocation, this reduces the number of allocation units, and hence the DP state count, by approximately a factor of d_{ff}/S .

Second, *accuracy–efficiency trade-off*: because neurons are sorted by sensitivity before partitioning, neurons within the same contiguous micro-expert have similar importance. Consequently, requiring them to share one bitwidth incurs little empirical loss of optimization headroom. The ablation in Section 4.3 (Figure 7) shows diminishing returns as S increases from 2 to 4 and then to 8. With the default $S = 8$, each micro-expert contains $d_{\text{ff}}/8$ hidden channels while retaining most of the observed accuracy benefit. Thus, micro-expert partitioning preserves the main benefit of fine-grained allocation at a substantially lower search cost.

Energy-based CAMERA-DP baseline. CAMERA-DP is the controlled baseline reported in the main experiments. It retains the same global DP search and micro-expert structure as DartMoQP, but replaces the quantization-loss-based sensitivity with CAMERA’s neuron-energy metric. Neurons are sorted by descending energy, and the block loss for the k -th micro-expert at bitwidth b is defined as

$$\text{block_loss}_{\text{energy}}[k][b] = \frac{\text{total_energy}[k]}{b + 1},$$

where $\text{total_energy}[k]$ is the sum of the CAMERA energy scores of the neurons in the micro-expert. The $+1$ in the denominator avoids division by zero at $b = 0$ (pruning) and sets the 0-bit loss equal to the total energy, giving the DP a simple pruning penalty proportional to unit importance.

Allocation Morphology Figure 5 contrasts uniform allocation with the skewed allocation produced by DartMoQP on an L2 expert of DeepSeek-v2-Lite. Under uniform allocation every micro-expert receives the same bitwidth; under DartMoQP, higher bitwidths are concentrated in high-sensitivity regions while low-sensitivity regions receive lower bitwidths or are pruned entirely (0-bit). This morphology is the structural signature of the budget transfer mechanism: at the same average bpw, bits saved on low-sensitivity micro-experts are moved to units with higher marginal gain.

A.5 Random Seed Fluctuation and Mixed Precision Suppression Effect

TurboQuant uses random orthogonal rotation for vector quantization, so its quantization error is affected by random seeds. The magnitude of this seed-induced variation, however, differs considerably across models: some models

exhibit substantial PPL fluctuations under uniform quantization, while others show almost none. This cross-model variation is itself informative, and points to a unified mechanism rooted in sensitivity heterogeneity.

The phenomenon mirrors the outlier channel problem in KV cache quantization research: a small number of high-sensitivity channels dominate overall distortion. In MoE expert weights, the same principle holds—when neuron sensitivity follows a strongly skewed distribution, a handful of high-sensitivity neurons contribute disproportionately to total quantization loss and are more susceptible to random rotation. Changes in their quantization errors across seeds amplify to the expert output level and become the main source of PPL variation. Conversely, when the sensitivity distribution is more uniform, random rotation errors are spread evenly across many neurons, and the net effect on PPL remains small.

From this perspective, micro-expert mixed precision interacts with seed variation not through a dedicated stability mechanism, but as a natural consequence of budget transfer: by allocating higher bitwidth to high-sensitivity micro-experts and lower bitwidth or 0-bit to low-sensitivity ones, DP search prioritizes the very units that drive both average loss and seed-induced fluctuation. In models where baseline fluctuation is large, this yields two benefits under the same average bpw: lower median PPL and narrower fluctuation ranges. In models where baseline fluctuation is already small, the primary effect is accuracy improvement, with little additional change in variance.

Beyond the eight-seed comparison in the main text (Table 4), Figure 6 summarizes an additional 32-seed analysis on DeepSeek-v1-MoE across different slicing granularities and mixed-precision configurations.

Uniform quantization produces a wide PPL distribution, reflecting high sensitivity to random rotation. After introducing micro-expert mixed precision, the median PPL decreases and both the interquartile and full ranges are substantially narrowed. This confirms that, when seed variation is driven by a skewed sensitivity distribution, differentiated bitwidth allocation mitigates fluctuation as a side effect of targeting marginal loss reduction.

A.6 Additional Experimental Results

FP16 and Uniform 8-bit Baselines Table 3 reports the FP16 and uniform 8-bit quantization baselines used to contextualize the main experimental results. The GPTQ-W8 and TurboQuant-W8 settings apply the two quantization backends used by DartMoQP, but do not use DartMoQP’s expert reordering, slicing, mixed-precision allocation, pruning, or dynamic-programming search. Together with the original unquantized FP16 models, these results provide common reference points for evaluating the gains of the main method.

For GPTQ-W8 and TurboQuant-W8, all attention, shared-FFN, and routed-expert linear projections are quantized uniformly to 8 bits; embeddings, normalization layers, and the language-model head remain in FP16. Both quantizers use a group size of 128 and preserve the original MoE architecture and routing. Quantization is calibrated using 64 WikiText2 sequences of length 2048 with seed 42. TurboQuant uses

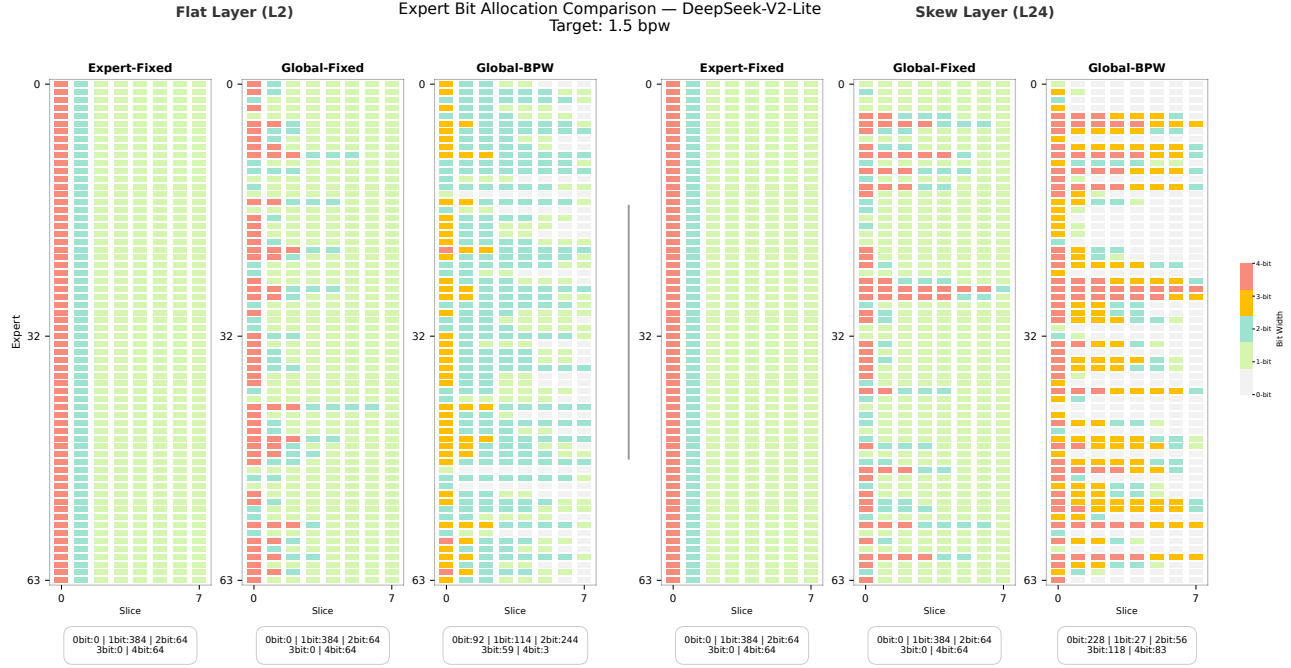


Figure 5: Bit allocation morphology on an L2 expert of DeepSeek-v2-Lite under TurboQuant (2.0 bpw). Uniform allocation assigns the same bitwidth to every micro-expert (top, flat). DartMoQP’s DP search concentrates higher bitwidths on high-sensitivity micro-experts and prunes low-sensitivity ones entirely (bottom, skewed), producing the characteristic non-increasing profile.

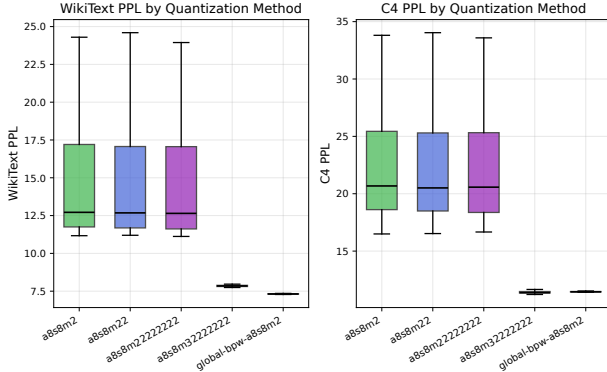


Figure 6: WikiText2 PPL distribution of DeepSeek-v1-MoE under 32 random seeds (TurboQuant, 1.5 bpw). Uniform quantization exhibits wide fluctuations; micro-expert mixed precision reduces both the median PPL and the spread as a side effect of protecting high-sensitivity units.

layer-wise QR rotations with seed $42 + l$, where l denotes the layer index. Reported downstream results use five-shot evaluation. ARC-Challenge, ARC-Easy, PIQA, and HellaSwag report normalized accuracy; the remaining tasks report accuracy. Avg. is the unweighted mean over the eight downstream tasks. Lower perplexity is better, whereas higher downstream accuracy is better.

Additional Model Results We present complete results for DeepSeekMoE-v1 and OLMoE-7B here, deferred from the main text due to space constraints. Detailed downstream comparisons at 1.0, 1.5, and 2.0 bpw are given in Tables 4–6.

A.7 Calibration Dataset Considerations

All experiments in this paper use the Wikipedia calibration dataset, which has become the de facto standard in the LLM quantization literature. GPTQ(Frantar et al. 2023), AWQ(Lin et al. 2023), and most subsequent work all calibrate on Wikipedia, which allows direct comparison with published baselines. We evaluate on C4 and downstream tasks that do not overlap with the calibration set, so validation data is not used to shape the quantization scheme.

The role of calibration data differs substantially across quantization paradigms, and understanding these differences helps contextualize our choice. GPTQ(Frantar et al. 2023) uses calibration activations to construct the Hessian and performs per-weight quantization with error compensation, so the calibration set directly shapes the final quantized weights. Exllama-V3 follows a similar but stronger dependence pattern, using Hessian-weighted error to guide tile-level trelis and codebook search, making it particularly sensitive to the source and scale of calibration data. Pure TurboQuant(Zandieh et al. 2025), by contrast, is data-agnostic at the quantization stage—the final quantized weights depend only on the original weights, the random rotation matrix, and the codebook, with no calibration data involved. Our TQ-IP variant falls in between: the quantization stage re-

Model	Setting	Wiki	C4	Avg.	ARC-C	ARC-E	PIQA	BoolQ	Wino	MNLI	Hella	MLLU
DSMoEv1	FP16	6.51	9.04	0.68	0.52	0.80	0.81	0.78	0.75	0.50	0.79	0.45
DSMoEv1	GPTQ-W8	6.51	9.05	0.67	0.53	0.80	0.80	0.78	0.75	0.50	0.79	0.45
DSMoEv1	TurboQuant-W8	6.51	9.05	0.67	0.52	0.79	0.81	0.78	0.74	0.50	0.79	0.45
DSv2-Lite	FP16	6.31	8.91	0.71	0.57	0.84	0.82	0.79	0.76	0.49	0.80	0.58
DSv2-Lite	GPTQ-W8	6.31	8.91	0.71	0.57	0.84	0.82	0.79	0.77	0.50	0.80	0.58
DSv2-Lite	TurboQuant-W8	6.31	8.91	0.70	0.57	0.84	0.82	0.79	0.76	0.50	0.80	0.58
Moonlight	FP16	7.12	10.36	0.74	0.63	0.86	0.82	0.82	0.75	0.52	0.81	0.70
Moonlight	GPTQ-W8	7.12	10.36	0.74	0.64	0.86	0.82	0.82	0.75	0.52	0.81	0.70
Moonlight	TurboQuant-W8	7.12	10.37	0.74	0.63	0.86	0.82	0.82	0.75	0.52	0.81	0.70
OLMoE	FP16	7.49	10.47	0.68	0.56	0.82	0.82	0.75	0.71	0.45	0.78	0.53
OLMoE	GPTQ-W8	7.50	10.47	0.68	0.55	0.82	0.82	0.75	0.72	0.45	0.78	0.53
OLMoE	TurboQuant-W8	7.50	10.47	0.68	0.56	0.82	0.82	0.75	0.72	0.45	0.78	0.53
Qwen3	FP16	8.71	12.15	0.80	0.68	0.88	0.83	0.89	0.70	0.82	0.78	0.80
Qwen3	GPTQ-W8	8.72	12.16	0.80	0.68	0.88	0.83	0.89	0.70	0.82	0.78	0.80
Qwen3	TurboQuant-W8	8.73	12.17	0.80	0.68	0.88	0.83	0.89	0.70	0.82	0.78	0.79

Table 3: FP16 and uniform 8-bit quantization baselines. Wino = Winogrande, Hella = HellaSwag.

Model	Method	Wiki ↓	C4 ↓	Avg. ↑	ARC-C	ARC-E	PIQA	BoolQ	Wino	MNLI	Hella	MLLU
DSMoEv1	GPTQ-Origin	132.71	566.14	0.35	0.26	0.26	0.50	0.38	0.53	0.36	0.26	0.27
DSMoEv1	TQ-Origin	663.68	723.96	0.35	0.25	0.27	0.52	0.38	0.53	0.36	0.26	0.25
DSMoEv1	GEMQ	$6.15 \times 10^7^\dagger$	$1.39 \times 10^8^\dagger$	0.38	0.27	0.25	0.51	0.62	0.52	0.36	0.27	0.26
DSMoEv1	CAMERA-DP	278.70	573.56	0.35	0.25	0.26	0.52	0.38	0.51	0.36	0.27	0.24
DSMoEv1	GPTQ-DP	10.88	18.56	0.52	0.38	0.65	0.69	0.63	0.62	0.40	0.55	0.27
DSMoEv1	IPE-TQ-DP	9.96	20.58	0.52	0.37	0.68	0.66	0.69	0.62	0.40	0.50	0.25
OLMoE	GPTQ-Origin	33766.75	18911.66	0.36	0.25	0.27	0.52	0.47	0.50	0.32	0.26	0.25
OLMoE	TQ-Origin	16508.11	8896.24	0.37	0.25	0.28	0.52	0.54	0.51	0.32	0.26	0.24
OLMoE	GEMQ	193.20	675.90	0.38	0.21	0.36	0.55	0.54	0.51	0.33	0.28	0.27
OLMoE	CAMERA-DP	16753.11	8156.68	0.37	0.26	0.29	0.52	0.57	0.50	0.32	0.26	0.26
OLMoE	GPTQ-DP	162.27	302.43	0.39	0.22	0.34	0.54	0.59	0.52	0.35	0.30	0.24
OLMoE	IPE-TQ-DP	22.59	42.14	0.48	0.34	0.57	0.63	0.62	0.58	0.37	0.43	0.29

Table 4: Additional model results at 1.0 bpw. Wino = Winogrande, Hella = HellaSwag.

[†]The GEMQ result on DSMoEv1 exhibits numerical divergence at this bitwidth and may not represent the method’s true capability; downstream tasks do not collapse due to the approximate nature of perplexity (details in Appendix A.9).

mains data-agnostic, but calibration data is used to estimate neuron sensitivity via activation-space inner-product distortion, which indirectly affects ranking, slicing, and bitwidth allocation.

Against this backdrop, our choice of a single calibration set (Wikipedia) serves three purposes. First, this work focuses on validating the unified framework and search algorithm rather than searching for the optimal calibration strategy, so a standard dataset suffices for methodological comparison. Second, aligning with the calibration setup used by GPTQ, AWQ, and follow-up work ensures that comparisons with published baselines are fair. Third, using distinct datasets for calibration (WikiText2) and evaluation (C4 and downstream benchmarks) avoids any risk of validation data leaking into the calibration process, which could inflate apparent performance.

We leave a systematic study of calibration data diversity, domain transfer, and sample efficiency across different quantizer paradigms as future work.

A.8 Industrial Deployment

The standard mixed-precision search yields globally optimal per-expert bit allocation, but deploying such highly personalized configurations in production inference frameworks introduces non-trivial engineering overhead: each expert requires its own quantization parameters, kernel dispatch path, and memory layout, which complicates batching, scheduling, and hardware utilization. We describe a simplified deployment scheme that retains nearly all accuracy benefits while drastically reducing deployment complexity, and report on our Triton-based kernel implementation that validates its practical feasibility.

Expert Merging and Memory Layout The key insight enabling efficient deployment is that the DP search already produces structurally regular allocations: within each expert, neurons are sorted by sensitivity and assigned non-increasing bitwidths, so high-bitwidth micro-experts are concentrated at the beginning of the weight matrix, low-bitwidth ones toward

Model	Method	Wiki ↓	C4 ↓	Avg. ↑	ARC-C	ARC-E	PIQA	BoolQ	Wino	MNLI	Hella	MMLU
DSMoEv1	GEMQ	10.24	20.00	0.50	0.34	0.64	0.66	0.66	0.60	0.38	0.47	0.25
DSMoEv1	CAMERA-DP	9.56	15.57	0.56	0.40	0.69	0.70	0.72	0.68	0.41	0.61	0.27
DSMoEv1	GPTQ-DP	8.74	13.67	0.57	0.40	0.69	0.74	0.62	0.68	0.43	0.66	0.31
DSMoEv1	IPE-TQ-DP	8.02	13.77	0.59	0.44	0.75	0.74	0.73	0.68	0.40	0.64	0.31
OLMoE	GEMQ	16.76	31.78	0.47	0.31	0.62	0.64	0.63	0.55	0.33	0.43	0.26
OLMoE	CAMERA-DP	33.46	46.65	0.52	0.38	0.61	0.68	0.65	0.59	0.40	0.54	0.29
OLMoE	GPTQ-DP	23.59	35.78	0.46	0.29	0.50	0.61	0.61	0.55	0.41	0.47	0.28
OLMoE	IPE-TQ-DP	15.10	22.78	0.58	0.44	0.69	0.71	0.69	0.66	0.44	0.59	0.38

Table 5: Additional model results at 1.5 bpw. Wino = Winogrande, Hella = HellaSwag.

Model	Method	Wiki ↓	C4 ↓	Avg. ↑	ARC-C	ARC-E	PIQA	BoolQ	Wino	MNLI	Hella	MMLU
DSMoEv1	GPTQ-Origin	8.62	12.91	0.58	0.43	0.72	0.76	0.67	0.70	0.36	0.72	0.32
DSMoEv1	TQ-Origin	11.76	16.50	0.52	0.34	0.67	0.70	0.68	0.59	0.37	0.53	0.26
DSMoEv1	CAMERA-GPTQ	8.27	12.70	0.59	0.43	0.72	0.75	0.69	0.68	0.41	0.70	0.35
DSMoEv1	CAMERA-TQ	7.98	11.79	0.62	0.46	0.74	0.75	0.77	0.69	0.45	0.69	0.35
DSMoEv1	GEMQ	7.22	11.96	0.60	0.45	0.77	0.77	0.70	0.70	0.40	0.67	0.32
DSMoEv1	CAMERA-DP	7.80	11.86	0.62	0.46	0.75	0.76	0.76	0.71	0.45	0.73	0.36
DSMoEv1	GPTQ-DP	7.99	12.30	0.60	0.44	0.75	0.78	0.67	0.70	0.42	0.73	0.36
DSMoEv1	IPE-TQ-DP	7.21	11.30	0.63	0.48	0.78	0.78	0.75	0.71	0.43	0.72	0.37
OLMoE	GPTQ-Origin	20.79	29.03	0.53	0.38	0.61	0.68	0.63	0.57	0.42	0.61	0.34
OLMoE	Origin-TQ	15.12	17.79	0.65	0.50	0.76	0.77	0.78	0.67	0.56	0.72	0.47
OLMoE	CAMERA-GPTQ	18.45	26.91	0.54	0.39	0.62	0.69	0.66	0.58	0.45	0.61	0.37
OLMoE	CAMERA-TQ	14.73	18.83	0.64	0.49	0.74	0.76	0.76	0.68	0.47	0.72	0.48
OLMoE	GEMQ	10.50	17.23	0.55	0.44	0.72	0.73	0.63	0.63	0.33	0.58	0.30
OLMoE	CAMERA-DP	17.28	22.75	0.61	0.47	0.73	0.72	0.75	0.65	0.50	0.68	0.41
OLMoE	GPTQ-DP	15.55	22.33	0.56	0.41	0.65	0.69	0.68	0.61	0.43	0.62	0.38
OLMoE	IPE-TQ-DP	12.20	17.19	0.63	0.50	0.75	0.76	0.74	0.67	0.52	0.70	0.45

Table 6: Additional model results at 2.0 bpw. Wino = Winogrande, Hella = HellaSwag.

the end, and pruned (0-bit) neurons are simply dropped. This regularity makes expert merging straightforward and lossless.

The merging procedure groups micro-experts of the same bitwidth within an expert into a single contiguous matrix block. Since the allocation is already sorted and non-increasing, each bitwidth level occupies a single contiguous region of the weight matrix, requiring no reordering or padding. The resulting memory layout is deployment-friendly: each expert consists of a small number of dense blocks (one per active bitwidth level), with pruned units removed entirely. This operation has zero impact on quantization accuracy because it only reorganizes storage, not the bit allocation itself.

Triton Kernel Implementation We have implemented a production-grade mixed-precision inference kernel based on Triton to validate the deployability of this scheme. The end-to-end pipeline follows scheme retrieval, expert merging, quantization, and kernel execution.

Compared with conventional MoE inference, the mixed-precision kernel introduces a bitwidth dimension in the parallelism hierarchy, while experts themselves are processed serially as usual. We adopt a 3D parallelization strategy: different bitwidth groups within an expert are scheduled in parallel, rotation groups within each bitwidth level are further paral-

lelized, and GEMM operations within each rotation group exploit fine-grained parallelism. This design maximizes hardware utilization under the mixed-bitwidth constraint, and the Triton programming model simplifies the implementation of this multi-level parallel structure. The current kernel is functionally complete but not yet fully optimized—particularly regarding load-balancing across bitwidth groups—and we leave more sophisticated scheduling strategies for future work.

One practical challenge arises from imbalanced bitwidth distribution within individual experts. When the number of neurons allocated to different bitwidths varies significantly, bit-level parallelism may suffer from load imbalance, leading to under-utilization of some computation lanes. This degradation is more pronounced in experts with highly skewed sensitivity distributions.

The full inference code, including Triton kernel implementations and the expert-merging dispatcher, is provided in the supplementary materials for reproducibility.

A.9 Analysis of GEMQ 1 bpw Divergence on DSMoEv1

At 1.0 bpw on DeepSeekMoE-v1, GEMQ produces numerically divergent perplexity (tens of millions on WikiText2 and over 100 million on C4) while downstream task accuracy

remains in a reasonable range. This apparent contradiction is not an artifact of measurement but a position-dependent failure mode of 1-bit expert quantization, which we analyze below.

Position-dependent error accumulation. We examine per-token negative log-likelihood (NLL) along the 2048-token evaluation sequences. For the first 1500 tokens, per-position NLL stays within a normal range comparable to other methods. Beyond approximately position 1500, however, the prediction distribution gradually collapses toward a fixed token, and per-token NLL rises sharply. Since perplexity is the exponential of mean NLL, even a modest number of high-NLL positions at the tail can inflate the overall PPL to extreme values. The mean NLL reaches approximately 18, which exponentiates to the tens-of-millions PPL values observed in Table 1 of the main paper.

This pattern is consistent with *error accumulation along context*: 1-bit quantization of MoE experts introduces large per-step distortion that compounds through the recurrent self-attention and FFN layers. Early in the sequence, residual representations are still close to the unquantized manifold, so predictions remain stable. As context grows, the accumulated quantization error drifts the internal representations away from their typical distribution, eventually causing the output logits to degenerate.

Disappearance above 1 bpw. The phenomenon is sharply bitwidth-dependent. At 1.125 bpw—a mere 12.5% increase in bit budget—the divergence disappears entirely: PPL drops to 16.61 (WikiText2) and 30.29 (C4), well within the normal range. This confirms that the divergence is a boundary effect of operating at exactly 1 bpw with expert-level allocation and no intra-expert pruning, not a fundamental limitation of GEMQ at slightly higher bitrates. It also illustrates why the ability to prune trivial units (0-bit) rather than force them to 1-bit is crucial at ultra-low bitwidths: 1-bit neurons that contribute nothing but error can be removed entirely, eliminating their contribution to position-dependent accumulation.

Why downstream tasks are less affected. The downstream benchmarks we evaluate consist primarily of multiple-choice questions with relatively short input contexts—well under the 1500-token threshold where divergence sets in. Since these tasks never enter the unstable tail region, the model’s predictions remain representative and accuracy does not collapse to the same degree as PPL. This explains the seemingly paradoxical observation that a method can have a PPL in the tens of millions while still achieving non-trivial downstream accuracy. We note that this PPL–task decoupling is specific to short-context evaluations; for long-context generation or retrieval tasks, the position-dependent divergence would likely cause more visible degradation.

We include GEMQ’s 1 bpw result in the main table for completeness and transparency, but caution against interpreting it as a representative measure of the method’s capability at higher bitrates.

A.10 Limitations and Future Work

Known Limitations Although DartMoQP performs well across the evaluated models and bitwidth ranges, several limitations are worth noting explicitly.

First, the continuous micro-expert constraint used by our DP search is an approximation. While sorting by sensitivity yields near-optimal allocations with only minor local crossovers, we do not provide a strict theoretical bound on the optimality gap, and the quality of the approximation may vary across architectures and bitwidth regimes.

Second, the log-domain quadratic fit and 0-bit extrapolation, while robust across most of our evaluated models, show a notable outlier on Qwen3-30B-A3B, which exhibits both higher violation rates and more severe underestimation of pruning loss. This suggests that larger or deeper models may have more complex distortion-rate behavior in the low-bit regime that the quadratic approximation does not fully capture. The underlying cause—whether it is model size, architectural choices, training dynamics, or a combination—remains an open question.

Third, the effect of mixed-precision allocation on seed-induced fluctuation varies across models. On models with strongly skewed sensitivity distributions, mixed precision substantially reduces TurboQuant’s PPL variance as a side benefit; on models with more uniform distributions, the effect is minimal. We have provided a qualitative explanation based on sensitivity heterogeneity, but a quantitative, predictive model of when and why fluctuation occurs across different model architectures is still lacking.

Future Directions Beyond these known limitations, several directions merit further investigation. Expert-level bpw budget allocation is currently set manually; developing automated allocation methods based on per-expert importance or layer-wise sensitivity profiles would further reduce manual tuning. Evaluating the framework on larger-scale MoE models—beyond the 7–30B parameter range studied here—would help establish whether the observed patterns, including both the quadratic fit and the Qwen3-30B outlier, scale predictably with model size. Finally, exploring co-optimization with hardware architectures and inference-specialized hardware could further improve the deployment efficiency of mixed-precision MoE models.